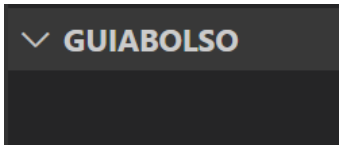
Guia de Bolso

Django Essentials

1) py –list

Esse comando lista todas as versões do Python instaladas.

Passo 1- crie uma pasta na área de trabalho com o nome guiabolso e abra no Visual Studio Code



Passo 2 – com o Python instalado, abra o terminal e digite o comando:

```
py --list
```

Veja o resultado:

```
-V:3.14 *        Python 3.14 (64-bit)
-V:3.13          Python 3.13 (64-bit)
-V:3.12          Python 3.12 (64-bit)
```

Caso queira conhecer toda a versão:

```
py -3.14 --version
```

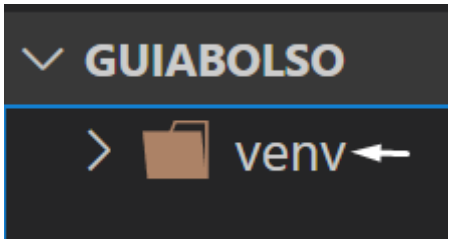
```
Python 3.14.3
```

2) py -3.x -m venv nome_do_env

Esse comando é para criar um ambiente virtual especificando uma versão.

```
py -3.14 -m venv venv
```

No explorer observe uma pasta que foi criada



Ative a venv

```
venv\scripts\activate
```

Caso precisar também pode desativar:

```
venv\scripts\deactivate
```

Veja a versão do python na venv :

```
\guiabolso>python --version
```

```
Python 3.14.3
```

3) pip install django==6.0.2

O comando pip install django vai instalar a dependência do Django responsável pelo framework. A partir de agora o terminal de comando vai receber comandos para criar projetos, criar apps, executar aplicações web, migrations para banco de dados entre outros.

- **Comando:** `django-admin startproject setup .`

- **O Pulo do Gato (O Ponto Final):** O `.` no final evita que o Django crie uma pasta dentro da outra, mantendo seu projeto organizado e limpo.

- **O Maestro:** O arquivo `manage.py` é o seu braço direito. Ele executará quase tudo daqui para frente.

Para verificar se foi instalado crie o primeiro projeto:

```
django-admin startproject setup .
```

Veja agora a sua estrutura de pastas:

Observe que será criada uma pasta **setup** e um arquivo **manage.py**

4) python manage.py runserver

Esse comando serve para executar um servidor local. Ao executar o comando irá aparecer:

```
You have 18 unapplied migration(s). Your project may not work properly until you apply the mig
rations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
March 06, 2026 - 11:53:25
Django version 6.0.2, using settings 'setup.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production
 WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/de
ployment/
```

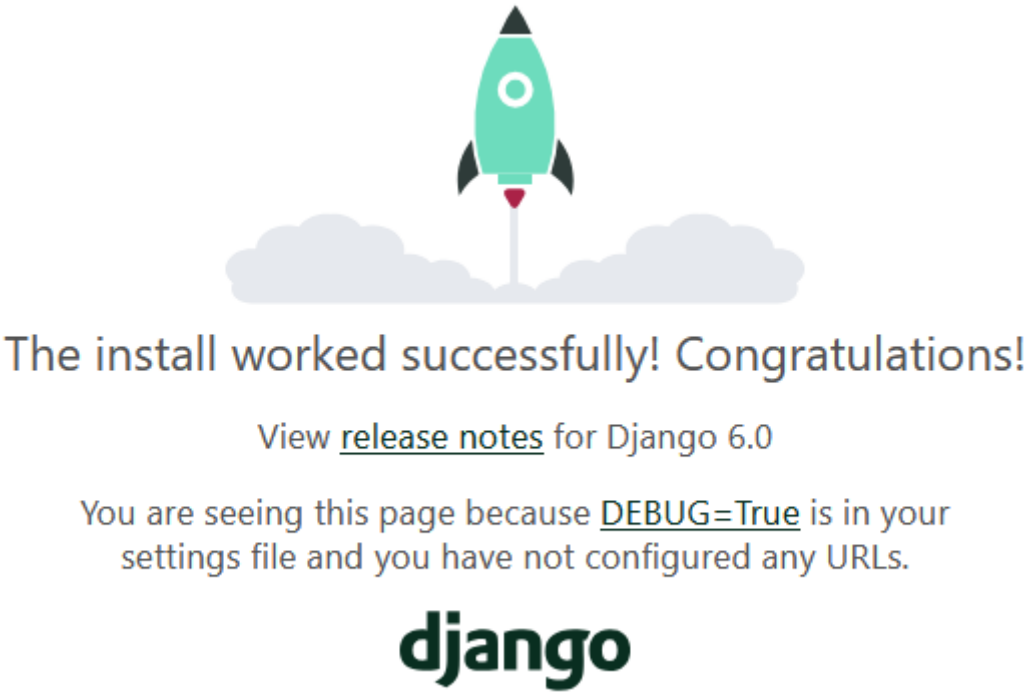
Você deve clicar em https://127.0.0.1:8000/

O localhost é esse. Veja abaixo o foguetinho, esse foguetinho representa que deu tudo certo na instalação do Django. Parabéns!

O "Foguetinho" da Vitória 🚀

Hora de ver o sistema ganhar vida!

- **Comando:** `python manage.py runserver`
- **O que observar:** O aviso em vermelho (18 migrations) é normal agora; são as tabelas padrão do Django esperando para serem criadas.
- **A URL:** Acesse `http://127.0.0.1:8000/`. Se o foguetinho aparecer, o motor do seu sistema de sindicalizados está ligado!



5) python manage.py startapp associado

Agora você irá precisar criar um aplicativo no Django. Dentro do framework ele entende assim:

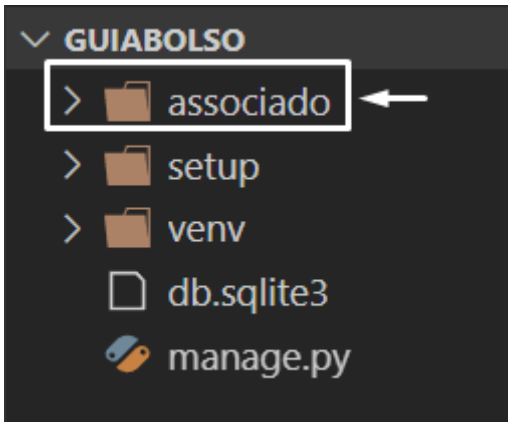
Ele tem um projeto. Esse projeto tem vários aplicativos. Cada aplicativo é um pedaço do projeto de forma modular e integrada. Cada aplicativo tem sua configuração de urls e views.

Abra o **Microsoft Visual Studio Code** e o terminal:

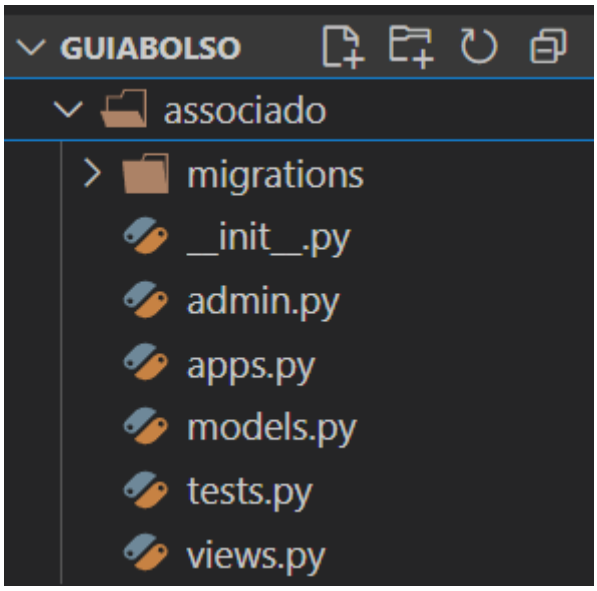
```
\guiabolso>python manage.py startapp associado
```

O objetivo do aplicativo é cadastrar o associado e depois gerar uma carteirinha para o mesmo.

Vamos ver o que foi gerado com esse comando:



Ao clicar na pasta do aplicativo:



6) Git init

O primeiro comando do git é realizado no terminal. Esse comando serve para inicializar o git.

```
\guiabolso>git init
```

7) Git Status

Esse comando verifica o que esta acontecendo com seus arquivos

```
\guiabolso>git status
```

Ele vai mostrar os arquivos que não estão gerenciados pelo git

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        associado/
        db.sqlite3
        manage.py
        setup/
```

8) Git add .

Para gerenciar os arquivos é necessário utilizar o git add .

Depois disso dê um git status e verá verde

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   associado/__init__.py
        new file:   associado/admin.py
        new file:   associado/apps.py
        new file:   associado/migrations/__init__.py
        new file:   associado/models.py
        new file:   associado/tests.py
```

Em todos os arquivos, pois agora estão gerenciados.

9) Git commit -m “mensagem”

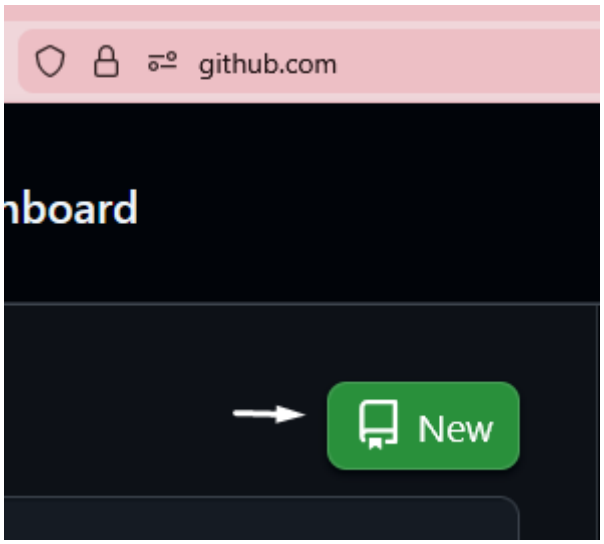
Esse comando é para salvar no repositório local

```
\guiabolso>git commit -m "salvando a primeira sessão do guia"
```

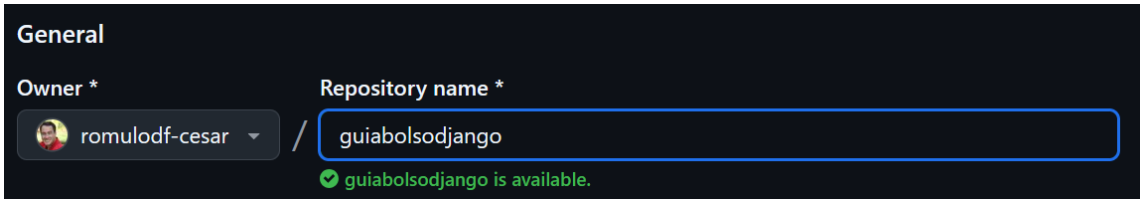
```
 18 files changed, 207 insertions(+)
 create mode 100644 associado/__init__.py
 create mode 100644 associado/admin.py
 create mode 100644 associado/apps.py
 create mode 100644 associado/migrations/__init__.py
 create mode 100644 associado/models.py
```

10) Git remote add

Primeiramente entre no github



Clique no botão verde para criar um novo repositório



Copie o comando git remote add

Copie no terminal

```
\guiabolso>git remote add origin https://github.com/romulodf-cesar/guiabolsodjango.git
```

Dê um comando git remote

```
\guiabolso>git remote
```

E veja o repositório remoto criado

```
origin
```

11) Git config –global

O objetivo é setar o usuário e email do github

```
\guiabolso>git config --global user.name "romulodf.cesar"

\guiabolso>git config --global user.email "romulodf.cesar@gmail.com"
```

12) Git push -u origin master

Esse comando vai enviar os arquivos do repositório local para o repositório remoto

```
\guiabolso>git push -u origin master
```

```
info: please complete authentication in your browser...
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 6 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (20/20), 5.71 KiB | 974.00 KiB/s, done.
Total 20 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/romulodf-cesar/guiabolsodjango.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

13) Git Clone

Ele é usado para clonar um projeto. Muito utilizado quando alguém compartilha o código e você faz o download do projeto. Isso mesmo, didaticamente falando o git clone é um download. O comando faz o donwload de todos os arquivos públicos do repositório remoto.

git clone **<< o endereço do repositório você deve colar aqui >>**

14) Pilares de Boas Práticas

Algumas boas práticas irei listar no nosso guia de bolso:

- Rotas inteligentes: organizar URLs diferentes para o cadastro, a lista de sócios e a geração da carteirinha.
- Arquivos estáticos (Static Files): aprender a carregar o CSS e as imagens (logos do sindicato, fotos) de forma específica e organizada.
- DRY (Don't Repeat Yourself): criar templates base para evitar código duplicado. Se a carteirinha mudar de layout, mudamos em um só lugar.

15) Segurança e Deploy

⚠ **Atenção do Professor:** Nunca suba sua `SECRET_KEY` ou senhas do banco de dados para o GitHub.

- Aprenderemos a usar **Variáveis de Ambiente** para manter os dados sensíveis do sindicato longe de olhos curiosos na nuvem.

🛠 Glossário Técnico (Django Style)

- `Templates`: O "esqueleto" HTML que o Django preenche com os dados do sindicalizado.
- `Static Files`: Arquivos de design (CSS, JS, Imagens) que não mudam.
- `Routes (URLs)`: Os caminhos que levam o usuário até a funcionalidade desejada.
- `Environment Variables`: Cofres onde guardamos as chaves secretas do projeto.

16) Django

O Django é um framework Python feito para **perfeccionistas com prazos apertados**. No nosso caso, isso significa criar o sistema do sindicato rápido, com segurança e sem "reinventar a roda".

- **Content-Driven:** Ele nasceu em um jornal, por isso é perfeito para gerenciar dados (como o cadastro de sócios).
- **Bateria Inclusa:** Já vem com banco de dados configurado, sistema de administração e segurança nativa.

17) Arquitetura MVT

Para organizar a carteirinha do sindicalizado, o Django separa as responsabilidades:

- **Model (Modelo):** Onde definimos os dados do sócio (Nome, CPF, Data de Filiação).
- **View (Visão):** A lógica que busca o sócio no banco e decide o que mostrar.
- **Template (Modelo Visual):** O HTML que o usuário vê (a cara da carteirinha)

18) Por que usar no Sindicato? (Vantagens Reais)

- **ORM (O Fim do SQL):** Você não precisa escrever códigos complexos de banco de dados para salvar um novo sócio. O Django traduz o Python para o Banco de Dados para você.
- **Admin Nativo:** O Django já te dá um "Painel de Controle" pronto para cadastrar, editar ou excluir sindicalizados sem você precisar criar uma tela para isso agora.
- **Segurança:** Proteção automática contra ataques (SQL Injection), garantindo que os dados dos sócios fiquem protegidos.
- **Fullstack:** Conseguimos fazer o verso (banco de dados) e a frente (carteirinha) no mesmo lugar.

Glossário Técnico (O Vocabulário do Prof. Rômulo)

- **CRUD:** As 4 operações básicas (Criar, Ler, Atualizar e Deletar) que faremos com os sócios.
- **ORM:** A ferramenta que liga seu código Python ao Banco de Dados.
- **MVT:** O padrão de organização do Django (**Model, View, Template**).
- **Open Source:** Ferramenta gratuita e mantida por uma comunidade mundial.

19) O Documento de Identidade do Projeto

Para que outros desenvolvedores (ou você mesmo no futuro) saibam o que o sistema do sindicato precisa para rodar, criamos o arquivo de dependências.

- **Comando:** `pip freeze > requirements.txt`
- **Por que fazer?** Ele lista as versões exatas do Django e outras bibliotecas. Sempre que instalar algo novo, repita esse comando!

```
pip freeze > requirements.txt
```

Veja que foi gerado o arquivo:

requirements.txt

Ao clicar no arquivo você verá as dependências instaladas:

```
requirements.txt U
requirements.txt
1  asgiref==3.11.1
2  Django==6.0.2
3  sqlparse==0.5.5
4  tzdata==2025.3
5
```

Tabela de Comandos de Gestão

Ação	Comando
Salvar Dependências	`pip freeze > requirements.txt`
Criar Estrutura	`django-admin startproject setup .`
Rodar o Servidor	`python manage.py runserver`
Ver Comandos	`django-admin help`

20) O Painel de Controle: `settings.py`

Este arquivo dentro da pasta `setup` é o coração das configurações.

- **Dica de Produtividade:** Use `Ctrl + B` (fechar lateral) e `Ctrl + J` (fechar terminal) para ter mais espaço de tela enquanto edita as configurações.

21) Deixando o Sistema "Brasileiro"

Para que o Django fale a nossa língua e entenda nosso relógio, alteramos as seguintes linhas (por volta da 106 e 108):

- **Idioma:** Alterar `LANGUAGE_CODE = 'en-us'` para `'pt-br'`.
  - *Resultado:* Mensagens de erro, botões do admin e a página inicial agora falam português.
- **Fuso Horário:** Alterar `TIME_ZONE = 'UTC'` para `'America/Sao_Paulo'`.
  - *Resultado:* O banco de dados registrará o horário oficial de Brasília.

22) Modernidade (Django 4+)

Fique atento à evolução! O Django 4 agora usa o pacote nativo do Python chamado `zoneinfo` para gerenciar as horas. Esqueça o antigo `pytz`; o Django moderno já faz o trabalho pesado de forma mais eficiente.

🔍 Checkpoint de Configuração (`setup/settings.py`)

Python

```
# Como deve ficar o seu código:
LANGUAGE_CODE = 'pt-br'

TIME_ZONE = 'America/Sao_Paulo'

USE_I18N = True # Internationalization (Tradução)
USE_TZ = True   # Time Zone (Fuso Horário)
```

Abra o Microsoft e coloque o código de configuração:

```
LANGUAGE_CODE = 'pt-br'

TIME_ZONE = 'America/Sao_Paulo'

USE_I18N = True

USE_TZ = True
```

Vamos rodar o projeto novamente:

```
python manage.py runserver
```

Clique no link do localhost

```
You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s)
ntenttypes, sessions.
Run 'python manage.py migrate' to apply them.
March 06, 2026 - 17:33:33
Django version 6.0.2, using settings 'setup.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI serv
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

```
Seguir o link (ctrl + clique)
http://127.0.0.1:8000/
```

Com o sistema traduzido e o relógio ajustado, o "foguetinho" agora fala português!

23) O Perigo da Chave Exposta

Todo projeto Django nasce com uma `SECRET_KEY` no arquivo `settings.py`.

- **Regra de Ouro:** Nunca suba essa chave para o GitHub. Ela deve ser única e secreta para garantir a criptografia do seu sistema de sindicalizados.

24) Blindando o Projeto com `python-dotenv`

Para esconder dados sensíveis, usamos "etiquetas" que apontam para um arquivo oculto no seu computador.

1. **Instalação:** `pip install python-dotenv`
2. **Atualizar lista:** `pip freeze > requirements.txt` (Sempre!).
3. **O Cofre (`.env`):** Crie um arquivo chamado `.env` na raiz do projeto e cole sua chave lá: `SECRET_KEY = 'sua-chave-secreta-aqui'`

Abra o Microsoft Visual Studio Code e posicione-se no terminal:

Derrube o servidor usando **CTRL+C**

```
pip install dotenv
```

Atualize a lista:

```
pip freeze > requirements.txt
```

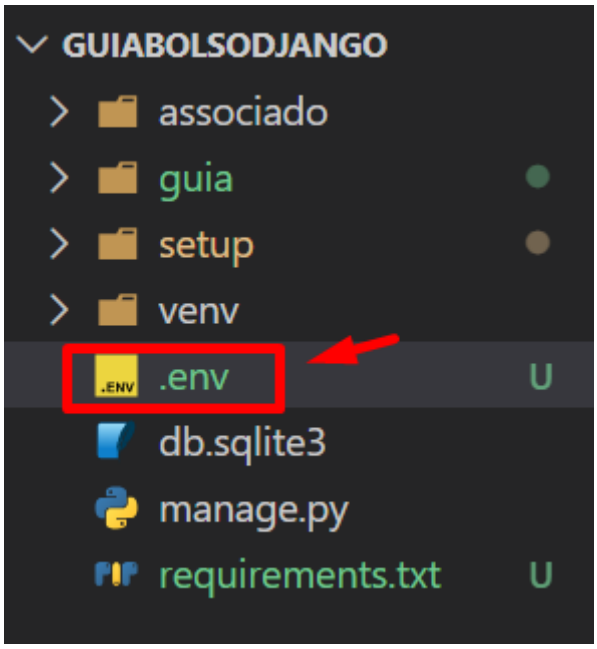
Use pip list

```
pip list
```

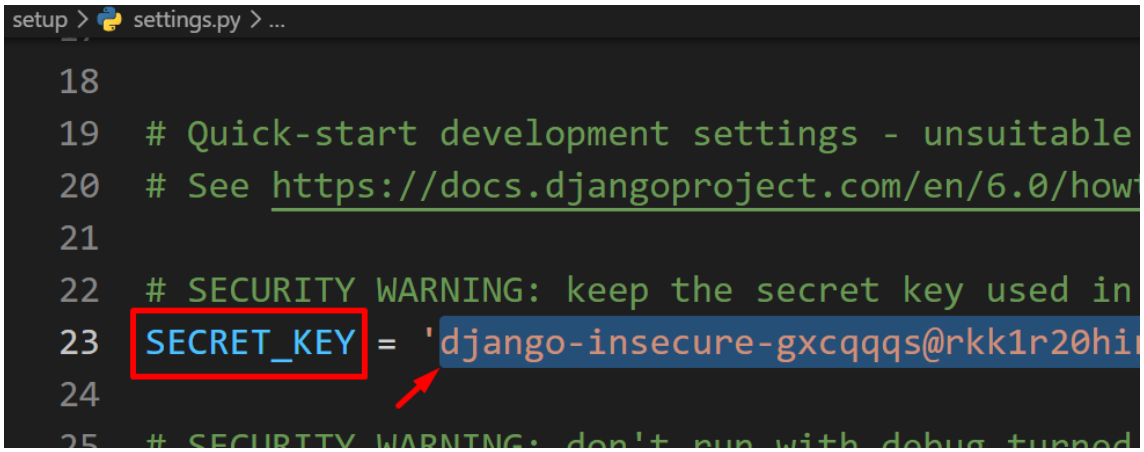
Visualização as **dependências**:

```
Package        Version
-------------- -------
asgiref        3.11.1
Django         6.0.2
dotenv         0.9.9
pip            26.0.1
python-dotenv  1.2.2
sqlparse       0.5.5
tzdata         2025.3
```

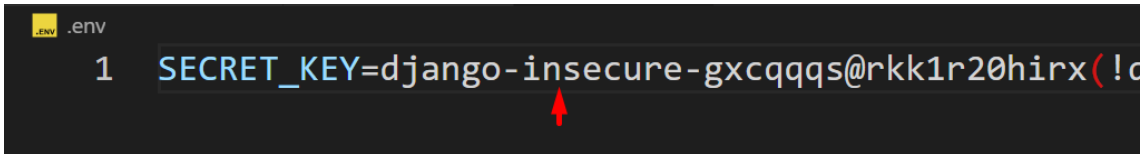
Agora crie o arquivo .env



Agora abra settings.py em setup e recorte a chave secreta:



Coloque a chave sem aspas simples ‘ ‘



25) Conectando o Cofre ao Django

No `settings.py`, precisamos dizer ao Django para ler esse arquivo `.env`:

- **No topo do arquivo:**

```
import os
from dotenv import load_dotenv
```

```
load_dotenv()
```

- **Na linha da Secret Key:**

Python
```
SECRET_KEY = str(os.getenv('SECRET_KEY'))
```

Deve ficar da seguinte forma:

```
from pathlib import Path
from dotenv import load_dotenv
import os

load_dotenv()

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


SECRET_KEY = ''
```

Na linha da SECRET_KEY coloque o seguinte código:

```
SECRET_KEY = str(os.getenv('SECRET_KEY'))
```

Por que isso funciona?

Quando você enviar o código para o GitHub, o arquivo `.env` ficará de fora (veremos como no próximo passo). O GitHub verá apenas o comando `os.getenv`, mas **não o valor real da chave**. O sistema do sindicato continua rodando porque, no seu computador, o Django consegue ler o arquivo local.

💻 Checklist de Segurança

Componente	Ação
Biblioteca	`pip install python-dotenv`
Arquivo Local	Criar `.env` (onde fica a chave real)
Código	Usar `os.getenv('SECRET_KEY')` no `settings.py`
Teste	Rodar `python manage.py runserver` e ver se o "foguetinho" ainda voa

26) O Que é o TCP?

Imagine o seu computador como um grande prédio comercial. O **TCP (Transmission Control Protocol)** é o recepcionista que garante que cada encomenda (informação) chegue à sala (aplicação) correta, sem extravios.

Portas de Comunicação

As portas são os "números das salas" dentro do seu IP. Sem elas, o computador não saberia se os dados que chegam da internet são para o seu **Navegador**, para o seu **E-mail** ou para o seu **Projeto Django**.

3. Tabela de Portas Cruciais

Para o desenvolvedor Django, estas são as portas que você encontrará no dia a dia:

Porta	Atribuição	Onde você verá isso?
80	**HTTP**	Navegação padrão (sites comuns).
443	**HTTPS**	Navegação segura (com cadeado).
8000	**Django Default**	A porta padrão que o Django usa para o seu servidor local.
22	**SSH**	Quando você for enviar o sistema do sindicato para o servidor real.
53	**DNS**	O sistema que traduz o nome do site em um endereço IP.

Exportar para as Planilhas

4. Por que o Django usa a 8000?

O Django evita as portas baixas (0 a 1024) porque elas são reservadas para o sistema operacional. Ao usar a **8000**, ele garante que o seu cadastro de sindicalizados não interfira no funcionamento básico do seu Windows ou Mac.

Dica do Professor:

Se você tentar rodar o `python manage.py runserver` e der erro dizendo que a porta já está em uso, você pode mudar a porta manualmente: `python manage.py runserver 8080`

Agora que já entendemos como a informação entra e sai do computador, estamos prontos para organizar o que vai (e o que NÃO vai) para o mundo exterior via GitHub. Vamos falar do `.gitignore`?

27) Gitignore

Entre no site: https://www.toptal.com/developers/gitignore

Na caixa digite Django e clique em Criar

Copie todo o arquivo que é gerado

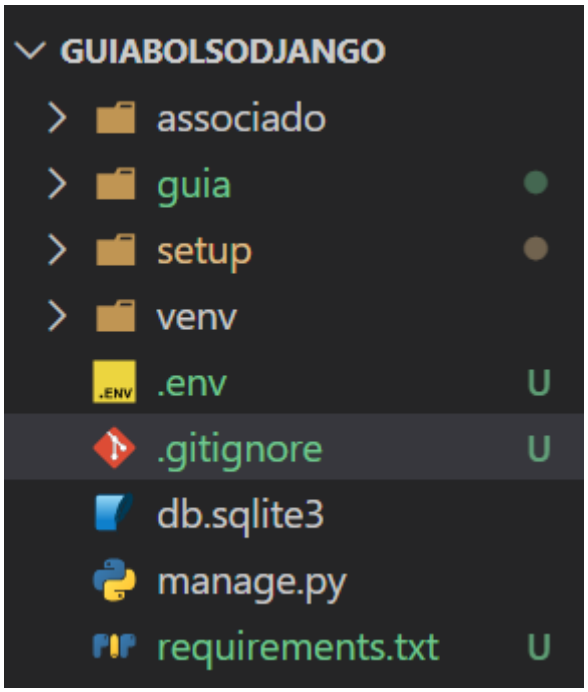
```
# Created by https://www.toptal.com/developers/gitignore/api/django
# Edit at https://www.toptal.com/developers/gitignore?templates=django

### Django ###
*.log
*.pot
*.pyc
__pycache__/
local_settings.py
db.sqlite3
db.sqlite3-journal
media

# If your build process includes running collectstatic, then you probably don't need or want to include staticfiles/
# in your Git repository. Update and uncomment the following line accordingly.
# <django-project-name>/staticfiles/

### Django.Python Stack ###
# Byte-compiled / optimized / DLL files
*.py[cod]
*$py.class
```

E coloque dentro de um arquivo que você irá criar na raiz do projeto chamado ".gitignore"



Faça o copiar e colar

28) O App "Associados"

O app associados nós já criamos vamos relembrar os comandos?

Diferente de um site de fotos genérico, o app **`associados`** será o motor que processa nomes, CPFs e gera as carteirinhas.

- **O Comando:** Primeiro, você parou o servidor (`Ctrl + C`) e criou o novo app com os seguintes comandos: `python manage.py startapp associados`

29) Registro no Grande Livro (`settings.py`)

O Django só vai reconhecer as funcionalidades do sindicato se você "der crachá" para o novo app.

- **Onde:** Vá em `setup/settings.py` -> Seção `INSTALLED_APPS`.
- **Ação:** Adicione `'associado',` ao final da lista.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'associado'
]
```

30) O Que Vamos Fazer Dentro Desse App?

Agora que a pasta `associados` existe, cada arquivo terá uma missão para o sindicato:

- **`models.py`:** Definir os campos da carteirinha (Nome, Matrícula, Foto).
- **`views.py`:** Criar a lógica que diz: "Se o sócio XPTO logar, mostre os benefícios dele".
- **`admin.py`:** Onde o administrador do sindicato vai aprovar novos cadastros.

💻 Checklist do Desenvolvedor

Passo	Comando / Ação
Criar App	`python manage.py startapp associados`
Registrar	Adicionar `'associados'` em `INSTALLED_APPS`

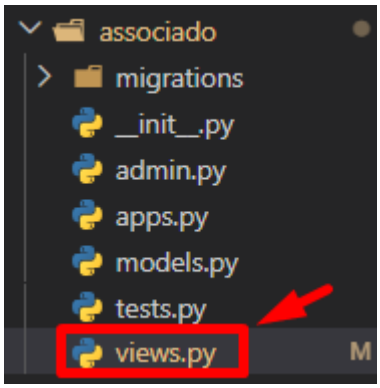
Passo	Comando / Ação
Testar	`python manage.py runserver`

Vamos ajustar tudo para a realidade do **SindApp**. O foco agora é tirar aquele foguete da frente e colocar a primeira mensagem de boas-vindas para os futuros sindicalizados.

31) Criando a Lógica (A View)

A **View** é quem decide o que o usuário vê. Para o nosso sistema de associados, vamos criar uma função simples que entrega uma resposta em HTML.

- **Onde:** `associado/views.py`



O código:

```
associado > views.py > index
1  from django.http import HttpResponse
2
3  def index(request):
4      return HttpResponse('<h1>SindApp</h1><p>Bem-vindo ao portal do associado!</p>')
```

from django.http import HttpResponse

def index(request):

return HttpResponse('<h1>SindApp</h1> Bem-vindo ao portal do associado! ')

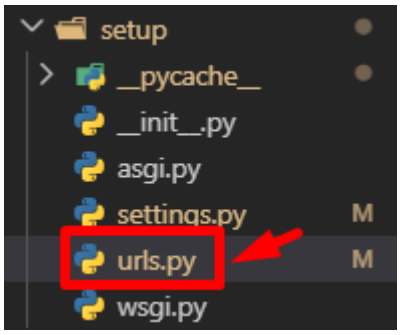
Termo Técnico: `HttpResponse` é a ferramenta que o Django usa para "responder" ao navegador com texto ou HTML puro.

32) Abrindo o Caminho (A URL)

Ter a View pronta não basta; o Django precisa saber em qual endereço (URL) ela deve aparecer. Precisamos ligar o endereço vazio `''` (a home) à nossa função `index`.

- **Onde:** `setup/urls.py`
- **Ação:** Importar a função e registrá-la na lista de caminhos

Clique em urls.py dentro de setup



Faça a importação da view index:

```
from django.contrib import admin
from django.urls import path
from associado.views import index

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

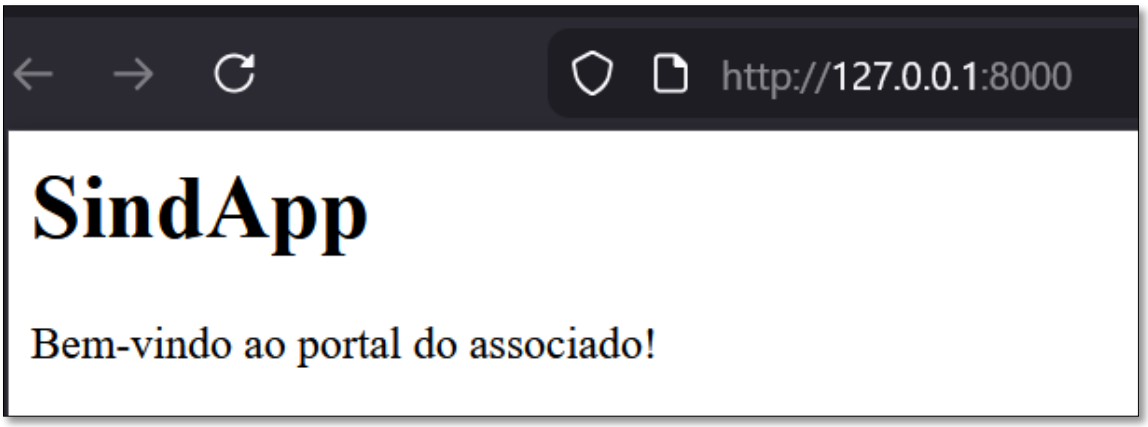
Agora coloque as aspas simples no primeiro parâmetro indicando que é a página home e o nome da view no segundo parâmetro:

```
from django.contrib import admin
from django.urls import path
from associado.views import index

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',index)
]
```

O caminho vazio representa a página inicial.

33) Teste de Fogo

Ao rodar o `python manage.py runserver` e acessar `http://127.0.0.1:8000/`, o foguetinho sumirá e você verá:

- **Título:** SindApp (em destaque H1).
- **Texto:** Bem-vindo ao portal do associado!